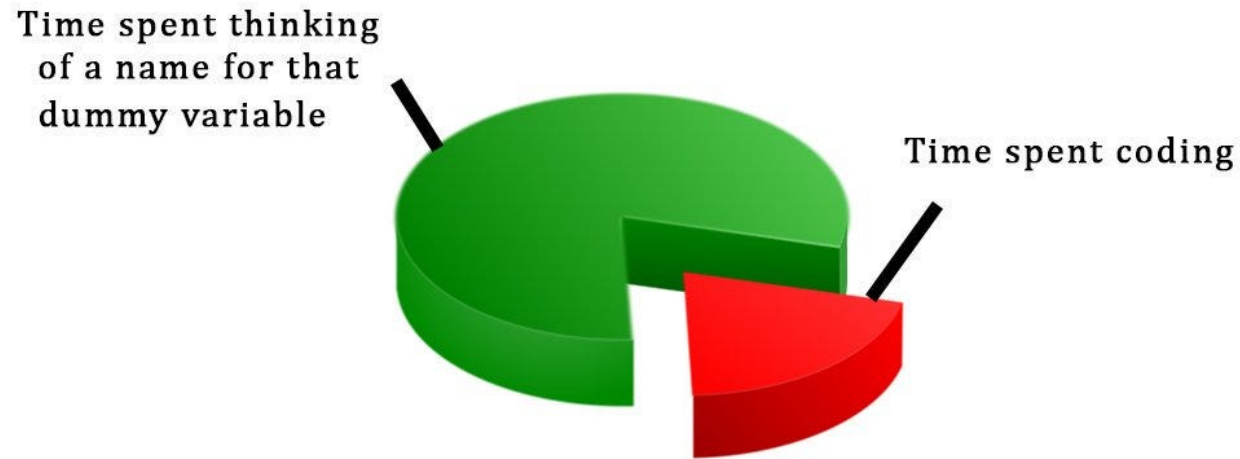


Programming Fact



Names, Bindings, and Scopes

SE 3002

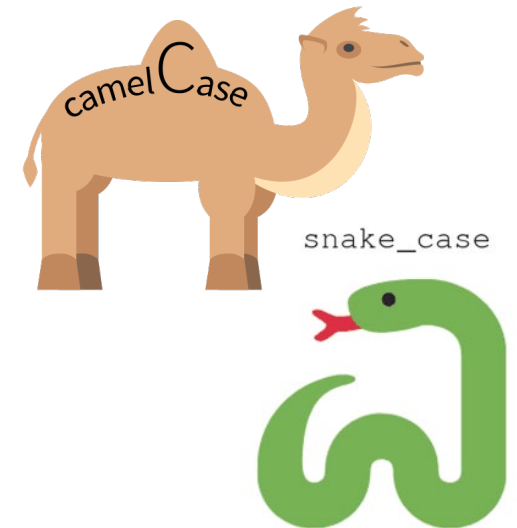
FUNDAMENTALS OF PROGRAMMING LANGUAGES

NAMES (IDENTIFIERS)

- Variables, subprograms, labels, classes, user-defined types, formal parameters
 - identifier (name) refers to a location where certain values can be stored
- Naming is about communication or efficiently getting the message across of your thoughts.
 - what will the other developer think when he reads this?
- Design issues for names:
 - What is the form of names?
 - What is the maximum length of a name?
 - Are names case-sensitive or not?
 - Are special words reserved words or keywords?

NAMES: FORM

- Names in most PL have the same form:
 - A letter followed by a string consisting of **letters, digits, and underscore** (`_`) characters
 - In some, they use **special characters** before a variable's name (`$` in Javascript, for more see slide 6)
- “**camelCase**” notation is more popular for various languages (with different variations)
 - Python, by contrast, recommends **snake case**, whereby words are instead separated by underscores (`_`), with all letters in lowercase (e.g., `preferred_first_name`)



Multiple-word identifier formats

Formatting	Name(s)
<code>two words</code>	flatcase ^{[13][14]}
<code>TWO WORDS</code>	UPPERFLATCASE ^[13]
<code>twoWords</code>	(lower) camelCase, dromedaryCase
<code>TwoWords</code>	PascalCase, UpperCamelCase, StudlyCase ^[15]
<code>two_words</code>	snake_case, pothole_case
<code>TWO_WORDS</code>	SCREAMING_SNAKE_CASE, MACRO_CASE, CONSTANT_CASE
<code>two_Words</code>	camel_Snake_Case
<code>Two_Words</code>	Pascal_Snake_Case
<code>two-words</code>	kebab-case, dash-case, lisp-case, spinal-case
<code>TWO-WORDS</code>	TRAIN-CASE, COBOL-CASE, SCREAMING-KEBAB-CASE
<code>Two-Words</code>	Train-Case, ^[13] HTTP-Header-Case ^[16]

NAMES: LENGTH

If too short

they cannot be easily remembered → **not readable**

- Using «X» for a counter or a flag.

If too long

they cannot be easily used → **not writable**

- «applicationSessionUserPermissionSynchronizationStatusUpdateHandler»

- Character limits for several languages:
 - **C99**: no limit but only the first 63 are significant; also, external names are limited to a maximum of 31
 - **C# and Java**: no limit, and all are significant
 - **C++**: no limit, but implementers often impose one

NAMES: SPECIAL CHARACTERS

- **PHP:** all variable names must begin with dollar signs (\$)
- **Perl:** all variable names begin with special characters (\$, @, %), which specify the variable's type
- **Ruby:** variable names that begin with @ are instance variables; those that begin with @@ are class variables

NAMES: CASE SENSITIVITY

- In many languages (e.g. C-based languages) uppercase and lowercase letters in names are distinct, e.g. rose, ROSE, Rose
- **Disadvantage:** not readable (names that look alike are different)

```
totalPrice = 100  
TotalPrice = totalPrice * 1.2  
totalprice = TotalPrice - 10  
  
print(totalprice)
```

- Worse in C++, Java, and C# because predefined names are mixed case (e.g. `IndexOutOfBoundsException`)

NAMES: CASE SENSITIVITY

Is it good for writability?

Also bad for writability

- A programmer has to remember the correct cases
 - Small typing differences cause bugs
- Slows down coding because developers must check case carefully

my folder : `Downloads`

me : `cd downloads`

Linux :



NAMES: SPECIAL WORDS

- A *reserved word* is a special word that cannot be used as a user-defined name (variable names, function names etc.)
 - for, while, int, etc.
- **Advantage:**
 - Compilers can **recognize program structure** faster because keywords always have fixed meanings
 - Improves program reliability
- **Disadvantage:** If there are too many, many collisions occur
 - e.g., COBOL has 300 reserved words!

CODING BEST PRACTICES

- **Do not use single-letter variables:** like x for the function *calculateSum*
 - Short variable names are only allowed for loop counters.
- **Avoid generic names:** such as “temp”, “test”, or “value”
 - Which value?
- **Use Plurality:** to refer to lists or arrays. It increases your code’s readability.

```
for (surname in name) {  
    print surname  
}
```



```
for (surname in surnames) {  
    print surname  
}
```

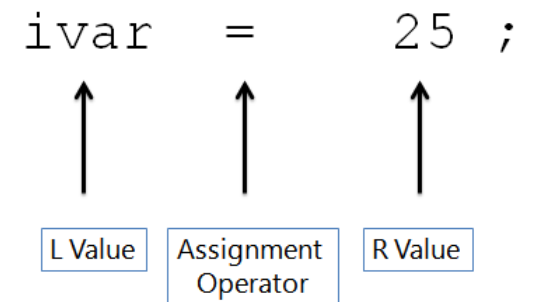


VARIABLES ATTRIBUTES

- Variables can be characterized by several attributes: **Name, Address, Value, Type, Lifetime, Scope**
- **Address** - the memory address with which it is associated
 - A variable may have **different addresses at different times** during execution
 - For a variable declared in a **recursive procedure**, in different steps of recursion it refers to different locations.
 - A variable may have **different addresses at different places** in a program
 - program can have two **subprograms sub1 and sub2 each of defines a local variable** that use the same name, e.g. sum
 - If two variable names can be used to access the same memory location, they are called **aliases**
 - Aliases are created via pointers, reference variables, C and C++ unions
 - Aliases are **harmful to readability** (program readers must remember all of them)

VARIABLES ATTRIBUTES (CONT.)

- **Type** - determines the range of values of variables and the set of operations that are defined for values of that type; in the case of floating point, type also determines the precision
 - `int` type in **Java** specifies a range of -2147483648 to 2147483647
- **Value** - the contents of the location with which the variable is associated
 - The l-value of a variable is its address (*appear on the left side of an assignment operator*)
 - The r-value of a variable is its value (*temporary value*)



THE CONCEPT OF BINDING

- A *binding* is an association between an **entity** and an **attribute**, such as between
 - a variable and its type or value,
 - an operation and a symbol
- *Binding time* is the time at which a binding takes place.
 - In C, `x` is bound to an `int` type in `compile time` when `int x = 5` is compiled.
 - In Python, we write `x = 5` without any type, then how do binding occur?

POSSIBLE BINDING TIMES

- **Language design time** -- bind operator symbols to operations
 - * is bound to the multiplication operation
- **Language implementation time**-- bind floating point type to a representation
 - int in C is bound to a range of possible values
- **Compile time** -- bind a variable to a type in C or Java
- **Load time** -- bind a C or C++ `static` variable to a memory cell
- **Runtime** -- bind a nonstatic local variable to a memory cell

STATIC AND DYNAMIC BINDING

- A binding is *static* if it first occurs **before run time** and remains **unchanged throughout program execution** (**C based, Java, Rust**).
 - **Fast and safe** but **not flexible**
- A binding is *dynamic* if it first occurs **during execution** or can change during execution of the program (**Python, Javascript, Ruby, C# (limited)**)
 - The advantages include **flexibility** and **no definite types declared** (PHP, JavaScript).
 - The disadvantages include adding to the **cost of implementation** (dynamic type checking and interpretation) and **type error detection by the compiler is difficult**

DYNAMIC TYPE BINDING

- Specified through an assignment statement e.g., JavaScript

```
list = [2, 4.33, 6, 8]; //1-D array
```

```
list = 17.3;           // float
```

EXPLICIT/IMPLICIT DECLARATION

- An *explicit declaration* is a program statement used for declaring the types of variables
 - Like in C based languages → `int x = 5;`
- An *implicit declaration* is a default mechanism for specifying types of variables through default conventions, rather than declaration statements
 - Python, Basic, Perl, Ruby, JavaScript, and PHP provide implicit declarations
 - **Advantage:** writability (a minor convenience)
 - **Disadvantage:** reliability (less trouble with Perl)

EXPLICIT/IMPLICIT DECLARATION

(CONTINUED)

- Some languages use **type inferencing** to determine types of variables (context)
 - C# - a variable can be declared with **var** and an initial value. **The initial value sets the type**
 - **Visual Basic 9.0+, ML, Haskell, and F#** use type inferencing. The context of the appearance of a variable determines its type

VARIABLE ATTRIBUTES (CONTINUED)

- Storage Bindings & Lifetime
 - **Allocation** - getting a cell from some pool of available cells
 - **Deallocation** - putting a cell back into the pool
- The lifetime of a variable is the time during which it is bound to a particular memory cell

CATEGORIES OF VARIABLES BY LIFETIMES

- **Static:** bound to memory cells **before execution begins** and **remains bound to the same memory cell throughout execution**
 - e.g., C and C++ `static` variables in functions
 - **Advantages:** efficiency (direct addressing), history-sensitive subprogram support
 - **Disadvantage:** lack of flexibility (no recursion)

CATEGORIES OF VARIABLES BY LIFETIMES

- **Stack-dynamic:** Storage bindings are created for variables in the runtime stack when their declaration statements are elaborated (*refers to the storage allocation*).
 - i.e. when the function is called
 - local variables in C subprograms (not declared `static`) and Java methods
- **Advantage:**
 - allows recursion;
 - All subprograms can share the same memory for their local variables.
- **Disadvantages:**
 - Overhead of allocation and deallocation
 - Subprograms cannot be history sensitive
 - Inefficient references (indirect addressing)

CATEGORIES OF VARIABLES BY LIFETIMES

- *Explicit heap-dynamic*: Allocated and deallocated by **explicit directives**, specified by the programmer, which take effect during execution (in runtime)
 - Referenced only through pointers or references, e.g. dynamic objects in C++ (via **new** and **delete**), all objects in Java

An example (C++):

```
int *intnode;           // Create a pointer
intnode = new int;      // Create the heap-dynamic variable
...
delete intnode;         // Deallocate the heap-dynamic variable
                        // to which intnode points
```

- **Advantage**: provides for dynamic storage management
- **Disadvantage**: inefficient and unreliable

CATEGORIES OF VARIABLES BY LIFETIMES

- *Implicit heap-dynamic*--Allocation and deallocation caused by assignment statements
 - all variables in APL; all strings and arrays in Perl, JavaScript, and PHP
 - Example (JavaScript): `highs = [74, 84, 86, 90, 71];`
- Advantage: flexibility (generic code)
- Disadvantages:
 - Inefficient, because all attributes are dynamic
 - Loss of error detection

VARIABLE ATTRIBUTES: SCOPE

- The *scope* of a variable is the *range of statements over which it is visible*
 - The *local variables* of a program unit are those that are declared in that unit
 - The *nonlocal variables* of a program unit are those that are visible in the unit but not declared there
 - ***Global variables*** are a special category of nonlocal variables
- The scope rules of a language determine how references to names are associated with variables

STATIC SCOPE

- To connect a name reference to a variable, **the compiler must find the declaration**
- **Search process:** search declarations, **first locally, then in increasingly larger enclosing scopes, until one is found for the given name**

- Two bindings for “x”
 - One of type `int`
 - Another `float`

An occurrence of “x”

```
int x;
void foo (int y)
{ float x;
  ... x ...
}
int main() {
  ...
  foo(x);
  // which binding of x?
  ...
}
```

STATIC SCOPE

```
1  #include <stdio.h>
2  float pi=3.1415; //This declaration is used
3  float area( float r){
4      return pi*r*r;
5  }
6  int main()
7  {
8      float pi=3; //This declaration is useless.
9      printf("area: %f",area(4)); //area: 50.26400
10
11     return 0;
12 }
```

In C

```
1  pi=3.1416           In Python
2  def area(r):
3      return pi*r*r
4  def area_print():
5      pi=3
6      print(area(4))
7
8  area_print() //50.26
```

Static scoping is used in all modern languages.

```
// A C program to demonstrate static scoping.
#include<stdio.h>
int x = 10;

// Called by g()
int f()
{
    return x;
}

// g() has its own variable
// named as x and calls f()
int g()
{
    int x = 20;
    return f();
}

int main()
{
    printf("%d", g());
    printf("\n");
    return 0;
}
```

Output = 10 ?